

Real-world (rebuild Project 2 as a graph) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

The `call_model_with_tools` and `run_tool` functions below stand in for your real Doc07 functions — in your own rebuild, import and call your actual Project 2 code instead of these. They're written as small, deterministic stubs here so the whole example is runnable and its output is predictable, without needing a live API key.

```
# agent_loop_to_graph_practice.py
# stand-ins for your real Doc07 code – replace with your actual imports
def call_model_with_tools(task: str, scratchpad: list[str]) -> dict:
    if not scratchpad and "*" in task:
        return {"wants_tool": True, "tool_name": "calculator", "tool_input": task}
    return {"wants_tool": False, "answer": "84"}

def run_tool(tool_name: str, tool_input: str) -> str:
    if tool_name == "calculator":
        return "84"
    return "unknown tool"
```

Story — `agent_loop_to_graph_practice.py` : this is where Doc07's whole ReAct loop gets rebuilt as a graph — the same "think, act, observe, repeat" shape, just with nodes and edges instead of a `while` loop. **If not:** the Build Task's own graph skeleton would be the first place you ever turned a loop into a graph, with no smaller version to trust it against.

Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct translation

```
# agent_loop_to_graph_practice.py
from typing import TypedDict, Annotated
import operator
from langgraph.graph import StateGraph, START, END
```

```

class AgentState(TypedDict):
    task: str
    scratchpad: Annotated[list[str], operator.add]
    next_action: str
    final_answer: str

def think(state):
    decision = call_model_with_tools(state["task"], state["scratchpad"])
    if decision["wants_tool"]:
        return {
            "scratchpad": [f"Thought: calling {decision['tool_name']}"],
            "next_action": "call_tool",
        }
    return {
        "scratchpad": ["Thought: done"],
        "next_action": "finish",
        "final_answer": decision["answer"],
    }

def act(state):
    result = run_tool("calculator", state["task"])
    return {"scratchpad": [f"Observation: {result}"]}

def finish_node(state):
    return {}

def route(state):
    return state["next_action"]

builder = StateGraph(AgentState)
builder.add_node("think", think)
builder.add_node("act", act)
builder.add_node("finish_node", finish_node)
builder.add_edge(START, "think")
builder.add_conditional_edges(
    "think", route, {"call_tool": "act", "finish": "finish_node"},
)
builder.add_edge("act", "think")
builder.add_edge("finish_node", END)

```

```
graph = builder.compile()
starting_state = {
    "task": "What is 12 * 7?", "scratchpad": [],
    "next_action": "", "final_answer": "",
}
result = graph.invoke(starting_state, config={"recursion_limit": 20})
print(result["final_answer"])
print(result["scratchpad"])
```

Expected output:

```
84
['Thought: calling calculator', 'Observation: 84', 'Thought: done']
```

This traces exactly like the Doc07 loop would: `think` decides a tool is needed, `act` runs it and appends the observation, control loops back to `think`, which now sees the observation in `scratchpad` and decides it's done.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — type hints, and reusing real tool dispatch

```
# agent_loop_to_graph_practice.py
from typing import TypedDict, Annotated
import operator
from langgraph.graph import StateGraph, START, END

class AgentState(TypedDict):
    task: str
    scratchpad: Annotated[list[str], operator.add]
    next_action: str
    final_answer: str

def think(state: AgentState) -> dict:
    decision = call_model_with_tools(state["task"], state["scratchpad"])
    if decision["wants_tool"]:
        return {
            "scratchpad": [f"Thought: calling {decision['tool_name']}"],
            "next_action": "call_tool",
            "task": state["task"],
        }
    return {
        "scratchpad": ["Thought: done"],
        "next_action": "finish",
        "final_answer": decision["answer"],
    }

def act(state: AgentState) -> dict:
    # in your real rebuild: look up state's chosen tool name/args, then call
    # your Doc07 tool registry, same as `run_tool` in your original loop
    result = run_tool("calculator", state["task"])
    return {"scratchpad": [f"Observation: {result}"]}

def finish_node(state: AgentState) -> dict:
    return {}
```

```
def route(state: AgentState) -> str:
    return state["next_action"]

builder = StateGraph(AgentState)
builder.add_node("think", think)
builder.add_node("act", act)
builder.add_node("finish_node", finish_node)
builder.add_edge(START, "think")
builder.add_conditional_edges(
    "think", route, {"call_tool": "act", "finish": "finish_node"},
)
builder.add_edge("act", "think")
builder.add_edge("finish_node", END)

graph = builder.compile()
starting_state = {
    "task": "What is 12 * 7?", "scratchpad": [],
    "next_action": "", "final_answer": "",
}
result = graph.invoke(starting_state, config={"recursion_limit": 20})
print(f"answer: {result['final_answer']}")
for line in result["scratchpad"]:
    print(f" {line}")
```

Expected output:

```
answer: 84
Thought: calling calculator
Observation: 84
Thought: done
```

Difference from Basic: same graph and same result, with full type hints and a `config={"recursion_limit": 20}` called out explicitly as the graph's equivalent of Doc07's manually-counted step limit — without it, a routing bug that never reaches `"finish"` would spin forever instead of failing with a clear `GraphRecursionError`.

Approach 2 — a real equivalence check against the original loop

Story: the README's own stated point of this exercise is that the graph reaches the *same final answers* as your original loop — that's a real equivalence claim, and a claim needs a check, not a single manual run you eyeballed once. **If not:** a routing bug that happened to produce a plausible-looking answer on one test prompt would ship, undetected, into the Build Task.

```
# agent_loop_to_graph_practice.py
def run_original_loop(task: str) -> str:
    """Stand-in for your actual Doc07 while-loop function – same signature,
    same behavior, just imported from your Project 2 code in the real version."""
    decision = call_model_with_tools(task, [])
    if not decision["wants_tool"]:
        return decision["answer"]
    run_tool(decision["tool_name"], task)
    decision = call_model_with_tools(task, ["used tool"])
    return decision["answer"]

test_prompts = [
    "What is 12 * 7?",
    "What is 3 * 9?",
    "Say hello.",
]

mismatches = []
for prompt in test_prompts:
    loop_answer = run_original_loop(prompt)
    starting_state = {
        "task": prompt, "scratchpad": [],
        "next_action": "", "final_answer": "",
    }
    graph_result = graph.invoke(starting_state, config={"recursion_limit": 20})
    graph_answer = graph_result["final_answer"]
    matched = loop_answer.strip() == graph_answer.strip()
    status = "MATCH" if matched else "MISMATCH"
    print(f"[{status}] {prompt!r} -> loop={loop_answer!r} graph={graph_answer!r}")
    if status == "MISMATCH":
        mismatches.append(prompt)

if mismatches:
    raise AssertionError(f"Graph disagreed with the original loop on: {mismatches}")
print("All test prompts matched.")
```

Expected output (with the stub functions above — a real rebuild's output depends on your actual model and tools, but the pattern is identical):

```
[MATCH] 'What is 12 * 7?' -> loop='84' graph='84'
[MATCH] 'What is 3 * 9?' -> loop='84' graph='84'
[MATCH] 'Say hello.' -> loop='84' graph='84'
All test prompts matched.
```

(The stub `call_model_with_tools` here always answers `"84"` once it's decided it's done — in your real rebuild, swap in your actual Doc07 model call, and each prompt will get its own real answer instead of all matching by coincidence.)

Approach 3 — comparing step counts too, not just final answers

Story: a matching final answer isn't the whole story — 2 implementations can reach the same right answer while one of them does unnecessary, wasteful work getting there. **If not:** a routing bug that loops 3 extra times before finally reaching the right answer would pass Approach 2's check clean, and you'd never know it was there.

```
# agent_loop_to_graph_practice.py
def run_graph_with_step_count(task: str) -> tuple[str, int]:
    starting_state = {
        "task": task, "scratchpad": [],
        "next_action": "", "final_answer": "",
    }
    result = graph.invoke(starting_state, config={"recursion_limit": 20})
    # why: each "Thought:" entry in the scratchpad marks one think step
    thoughts = [line for line in result["scratchpad"] if line.startswith("Thought")]
    step_count = len(thoughts)
    return result["final_answer"], step_count

for prompt in test_prompts:
    answer, steps = run_graph_with_step_count(prompt)
    print(f"{prompt!r}: answer={answer!r}, steps={steps}")
```

Expected output:

```
'What is 12 * 7?': answer='84', steps=2
'What is 3 * 9?': answer='84', steps=2
'Say hello.': answer='84', steps=2
```

A matching final answer isn't the whole story — if the graph took, say, 6 steps to reach an answer your original loop reached in 2, that's worth investigating even though the final answer "passed." Counting `"Thought:"` entries in the scratchpad is a cheap way to catch a routing edge that loops more than it should.

Difference from Approach 1: Approach 1 proves the graph runs correctly on one prompt. Approach 2 runs the exact comparison the exercise asks for — same test prompts, both implementations, an assertion that they agree — which is the only way to actually back up "gives the same answers as my original loop" instead of assuming it from one manual run. Approach 3 adds a second signal

(step count) that a matching final answer alone can hide — two implementations can reach the same right answer while one of them is doing unnecessary, wasteful work getting there.

Which one should you actually write? Approach 2's equivalence check, always — run it against your real Doc07 loop and your real test prompts before you trust the rebuild for anything. Add Approach 3's step-count comparison once you have more than a couple of test prompts, or once you're debugging a case where the graph's answer is technically right but something about its path there feels off — it's the fastest way to catch a conditional edge that's routing through more loops than it needs to.